

AHDS - Git & contrôle de version pour entretiens d'ingénierie

Git & Version Control for Engineering Interviews - Français / English

Questions/réponses courtes pour ingénieurs hardware, firmware et test. Le document couvre le workflow quotidien, configuration, dépôts distants, branches, merge, annulation/récupération, tags/releases, submodules, bisect, stash, hooks, Git LFS, worktrees et maintenance. ★ = question complémentaire ajoutée pour l'entretien / additional interview question.

1. Fondamentaux Git / Git Fundamentals

1. FR - ★ Qu'est-ce que Git ?

Réponse : Git est un système distribué de contrôle de version qui suit les modifications de fichiers et permet de travailler avec branches, commits et dépôts distants.

EN - ★ What is Git?

Answer: Git is a distributed version-control system that tracks file changes and supports branches, commits, and remote repositories.

2. FR - ★ Pourquoi utiliser Git dans un projet hardware/firmware ?

Réponse : Pour tracer les versions de firmware, scripts de test, documentation, configurations et fichiers de projet tout en conservant un historique des changements.

EN - ★ Why use Git in a hardware/firmware project?

Answer: To track firmware, test scripts, documentation, configuration, and project files while keeping a history of changes.

3. FR - ★ Qu'est-ce qu'un repository ?

Réponse : Un dépôt contient les fichiers suivis et l'historique Git du projet.

EN - ★ What is a repository?

Answer: A repository contains tracked project files and the Git history.

4. FR - Qu'est-ce que le dossier `.git` ?

Réponse : C'est le dossier caché contenant la base de données, les références et la configuration locale du dépôt.

EN - What is the `.git` directory?

Answer: It is the hidden directory containing the repository database, references, and local configuration.

5. FR - ★ Qu'est-ce qu'un commit ?

Réponse : Un commit enregistre un état logique des modifications indexées avec auteur, date, message et identifiant.

EN - ★ What is a commit?

Answer: A commit records a logical snapshot of staged changes with author, date, message, and identifier.

6. FR - ★ Qu'est-ce qu'un hash de commit ?

Réponse : C'est l'identifiant calculé du commit, utilisé pour le référencer de manière précise.

EN - ★ What is a commit hash?

Answer: It is the computed identifier of a commit, used to reference it precisely.

7. FR - ★ Qu'est-ce que `HEAD` ?

Réponse : `HEAD` désigne normalement le commit actuellement checkouté, souvent via la branche active.

EN - ★ What is `HEAD`?

Answer: `HEAD` normally points to the currently checked-out commit, usually through the active branch.

8. FR - ★ Quelle différence entre Git, GitHub et GitLab ?

Réponse : Git est l'outil de contrôle de version ; GitHub et GitLab sont des plateformes d'hébergement et de collaboration autour de dépôts Git.

EN - ★ What is the difference between Git, GitHub, and GitLab?

Answer: Git is the version-control tool; GitHub and GitLab are repository hosting and collaboration platforms.

2. Installation et configuration initiale / Installation and Initial Configuration

9. FR - ★ Comment installer Git sur Windows ?

Réponse : On peut utiliser l'installateur officiel Git for Windows, puis vérifier l'installation dans cmd, PowerShell ou Git Bash.

EN - ★ How do you install Git on Windows?

Answer: Use the official Git for Windows installer, then verify it in cmd, PowerShell, or Git Bash.

10. FR - ★ Comment vérifier que Git est installé ?

`git --version`

Réponse : Avec `git --version`.

EN - ★ How do you verify that Git is installed?

`git --version`

Answer: Use `git --version`.

11. FR - Que fait `git config --global user.name` ?

`git config --global user.name "Votre Nom"`

Réponse : Il définit le nom d'auteur utilisé par défaut pour les commits de cet utilisateur.

EN - What does `git config --global user.name` do?

`git config --global user.name "Votre Nom"`

Answer: It sets the default author name used for commits by that user.

12. FR - Que fait `git config --global user.email` ? <code>git config --global user.email "you@example.com"</code> Réponse : Il définit l'adresse email enregistrée dans les commits.	EN - What does `git config --global user.email` do? <code>git config --global user.email "you@example.com"</code> Answer: It sets the email address recorded in commits.
13. FR - Pourquoi utiliser la même adresse email que sur GitHub/GitLab ? Réponse : Cela facilite l'association des commits à votre compte sur la plateforme.	EN - Why use the same email as GitHub/GitLab? Answer: It helps the platform associate commits with your account.
14. FR - Que fait `git config --global init.defaultBranch main` ? <code>git config --global init.defaultBranch main</code> Réponse : Il configure `main` comme nom par défaut des nouvelles branches initiales.	EN - What does `git config --global init.defaultBranch main` do? <code>git config --global init.defaultBranch main</code> Answer: It configures `main` as the default initial branch name for new repositories.
15. FR - Que fait `git config --global core.autocrlf input` ? <code>git config --global core.autocrlf input</code> Réponse : Il convertit CRLF vers LF lors du commit, tout en laissant les LF intacts lors du checkout ; c'est courant sous Linux/macOS.	EN - What does `git config --global core.autocrlf input` do? <code>git config --global core.autocrlf input</code> Answer: It converts CRLF to LF on commit while leaving LF unchanged on checkout; it is common on Linux/macOS.
16. FR - ★ Comment afficher la configuration Git courante ? <code>git config --list</code> Réponse : Avec `git config --list` ; on peut aussi interroger une clé précise.	EN - ★ How do you display the current Git configuration? <code>git config --list</code> Answer: Use `git config --list` ; you can also query a specific key.

3. Création, clonage et connexion au dépôt distant / Creating, Cloning, and Connecting to a Remote

17. FR - Que fait `git init` ? <code>git init</code> Réponse : Il initialise un dépôt Git local dans le dossier courant et crée `.git`.	EN - What does `git init` do? <code>git init</code> Answer: It initializes a local Git repository in the current folder and creates `.git`.
18. FR - Que fait `git clone <URL>` ? <code>git clone <repository_URL></code> Réponse : Il copie un dépôt distant existant vers le PC local avec son historique.	EN - What does `git clone <URL>` do? <code>git clone <repository_URL></code> Answer: It copies an existing remote repository to the local PC with its history.
19. FR - Que fait `git remote add origin <URL>` ? <code>git remote add origin <repository_URL></code> Réponse : Il ajoute un dépôt distant nommé `origin` au dépôt local.	EN - What does `git remote add origin <URL>` do? <code>git remote add origin <repository_URL></code> Answer: It adds a remote repository named `origin` to the local repository.
20. FR - ★ Pourquoi le remote s'appelle-t-il souvent `origin` ? Réponse : `origin` est une convention courante pour le dépôt distant principal, mais le nom peut être différent.	EN - ★ Why is a remote often called `origin`? Answer: `origin` is a common convention for the main remote repository, but the name can be different.
21. FR - ★ Comment afficher les remotes configurés ? <code>git remote -v</code> Réponse : Avec `git remote -v`.	EN - ★ How do you display configured remotes? <code>git remote -v</code> Answer: Use `git remote -v`.
22. FR - Que fait `git push -u origin main` ? <code>git push -u origin main</code> Réponse : Il pousse la branche locale `main` vers `origin/main` et mémorise cette branche distante comme upstream.	EN - What does `git push -u origin main` do? <code>git push -u origin main</code> Answer: It pushes local `main` to `origin/main` and records that remote branch as the upstream.
23. FR - ★ Quelles méthodes courantes permettent de s'authentifier auprès d'un serveur Git ? Réponse : Typiquement HTTPS avec token/credential manager ou SSH avec paire de clés, selon la politique de l'entreprise.	EN - ★ What common methods are used to authenticate to a Git server? Answer: Typically HTTPS with a token/credential manager or SSH with a key pair, depending on company policy.
24. FR - ★ Pourquoi SSH est-il pratique pour Git ? Réponse : Une fois la clé configurée, il permet une authentification sécurisée sans retaper constamment un mot de passe.	EN - ★ Why is SSH convenient for Git? Answer: Once the key is configured, it provides secure authentication without repeatedly entering a password.

4. Cycle de travail principal et Staging Area / Main Workflow and Staging Area

25. FR - Que fait `git status` ? <code>git status</code> Réponse : Il affiche l'état du working tree et de l'index : fichiers modifiés, staged, supprimés ou non suivis.	EN - What does `git status` do? <code>git status</code> Answer: It shows the state of the working tree and index: modified, staged, deleted, or untracked files.
26. FR - ★ Qu'est-ce que la Staging Area ? Réponse : C'est l'index contenant exactement les modifications prévues pour le prochain commit.	EN - ★ What is the Staging Area? Answer: It is the index containing exactly the changes planned for the next commit.
27. FR - Que fait `git add <fichier>` ? <code>git add main.c</code> Réponse : Il ajoute les modifications du fichier sélectionné à la Staging Area.	EN - What does `git add <file>` do? <code>git add main.c</code> Answer: It stages the selected file's changes.
28. FR - Que fait `git add .` ? <code>git add .</code> Réponse : Il ajoute les modifications et nouveaux fichiers du répertoire courant et de ses sous-répertoires à l'index.	EN - What does `git add .` do? <code>git add .</code> Answer: It stages changes and new files from the current directory and its subdirectories.
29. FR - Que fait `git add -p` ? <code>git add -p</code> Réponse : Il permet de sélectionner interactivement les portions de modifications à mettre en staging.	EN - What does `git add -p` do? <code>git add -p</code> Answer: It lets you interactively choose which change hunks to stage.
30. FR - Que fait `git commit -m 'message'` ? <code>git commit -m "Fix ADC scaling"</code> Réponse : Il crée un commit avec les modifications actuellement staged et le message fourni.	EN - What does `git commit -m 'message'` do? <code>git commit -m "Fix ADC scaling"</code> Answer: It creates a commit from the currently staged changes using the supplied message.
31. FR - ★ Pourquoi éviter un commit énorme contenant plusieurs sujets différents ? Réponse : Des commits petits et cohérents sont plus faciles à relire, tester, revert et cherry-pick.	EN - ★ Why avoid one huge commit containing unrelated changes? Answer: Small, coherent commits are easier to review, test, revert, and cherry-pick.
32. FR - ★ Que signifie un bon message de commit ? Réponse : Il décrit clairement ce qui a changé et idéalement pourquoi, avec une portée suffisamment précise.	EN - ★ What makes a good commit message? Answer: It clearly describes what changed and ideally why, with a sufficiently specific scope.

5. Historique et différences / History and Differences

33. FR - Que fait `git log` ? <code>git log</code> Réponse : Il affiche l'historique des commits avec leurs détails.	EN - What does `git log` do? <code>git log</code> Answer: It displays commit history with details.
34. FR - Que fait `git log --oneline` ? <code>git log --oneline</code> Réponse : Il affiche un résumé compact avec un commit par ligne.	EN - What does `git log --oneline` do? <code>git log --oneline</code> Answer: It displays a compact history with one commit per line.
35. FR - Que fait `git diff` ? <code>git diff</code> Réponse : Il montre généralement les modifications du working tree qui ne sont pas encore staged.	EN - What does `git diff` do? <code>git diff</code> Answer: It generally shows working-tree changes that have not yet been staged.
36. FR - ★ Comment voir les modifications déjà staged ? <code>git diff --staged</code> Réponse : Avec <code>`git diff --staged`</code> ou <code>`git diff --cached`</code> .	EN - ★ How do you see already staged changes? <code>git diff --staged</code> Answer: Use <code>`git diff --staged`</code> or <code>`git diff --cached`</code> .

37. FR - Que fait `git log --author='Nom'` ? <code>git log --author="Nom"</code> Réponse : Il filtre l'historique par auteur.	EN - What does `git log --author='Name'` do? <code>git log --author="Nom"</code> Answer: It filters history by author.
38. FR - Que fait `git log --since='2 weeks ago'` ? <code>git log --since="2 weeks ago"</code> Réponse : Il affiche les commits plus récents que la période indiquée.	EN - What does `git log --since='2 weeks ago'` do? <code>git log --since="2 weeks ago"</code> Answer: It displays commits newer than the specified period.
39. FR - Que fait `git log -S 'texte'` ? <code>git log -S "function_name"</code> Réponse : Il cherche les commits qui ont modifié le nombre d'occurrences de ce texte dans le contenu.	EN - What does `git log -S 'text'` do? <code>git log -S "function_name"</code> Answer: It finds commits that changed the number of occurrences of that text in the content.
40. FR - Que permet `git log -L :func:file.c` ? <code>git log -L :func_name:src/main.c</code> Réponse : Il suit l'historique d'une fonction ou plage de lignes dans un fichier pris en charge.	EN - What does `git log -L :func:file.c` allow? <code>git log -L :func_name:src/main.c</code> Answer: It traces the history of a function or line range in a supported file.
6. Branches et fusion / Branches and Merging	
41. FR - Que fait `git branch` ? <code>git branch</code> Réponse : Il liste les branches locales et indique la branche active.	EN - What does `git branch` do? <code>git branch</code> Answer: It lists local branches and indicates the active branch.
42. FR - Que fait `git branch <nom>` ? <code>git branch feature_adc</code> Réponse : Il crée une nouvelle branche sans y basculer.	EN - What does `git branch <name>` do? <code>git branch feature_adc</code> Answer: It creates a new branch without switching to it.
43. FR - Que fait `git switch <branche>` ? <code>git switch main</code> Réponse : Il bascule vers la branche spécifiée.	EN - What does `git switch <branch>` do? <code>git switch main</code> Answer: It switches to the specified branch.
44. FR - Que fait `git switch -c <branche>` ? <code>git switch -c feature_uart</code> Réponse : Il crée une nouvelle branche puis bascule immédiatement dessus.	EN - What does `git switch -c <branch>` do? <code>git switch -c feature_uart</code> Answer: It creates a new branch and immediately switches to it.
45. FR - Quelle ancienne commande peut aussi changer de branche ? Réponse : `git checkout <branche>`; `git switch` est souvent plus explicite pour ce cas.	EN - What older command can also switch branches? Answer: `git checkout <branch>`; `git switch` is often clearer for this purpose.
46. FR - Que fait `git merge <branche>` ? <code>git merge feature_uart</code> Réponse : Il fusionne la branche spécifiée dans la branche courante.	EN - What does `git merge <branch>` do? <code>git merge feature_uart</code> Answer: It merges the specified branch into the current branch.
47. FR - ★ Qu'est-ce qu'un merge conflict ? Réponse : Git ne peut pas déterminer automatiquement quelle version conserver pour certaines modifications concurrentes.	EN - ★ What is a merge conflict? Answer: Git cannot automatically determine which version to keep for some competing changes.
48. FR - ★ Comment résoudre un merge conflict ? Réponse : Ouvrir les fichiers en conflit, choisir/corriger le contenu, tester, faire `git add` puis terminer le merge par un commit.	EN - ★ How do you resolve a merge conflict? Answer: Open conflicted files, choose/fix the content, test, run `git add`, then complete the merge with a commit.
49. FR - Que fait `git merge --no-ff <branche>` ? <code>git merge --no-ff feature</code> Réponse : Il force un commit de merge même lorsqu'un fast-forward serait possible, afin de préserver visuellement la branche dans l'historique.	EN - What does `git merge --no-ff <branch>` do? <code>git merge --no-ff feature</code> Answer: It forces a merge commit even when a fast-forward is possible, preserving the feature branch in history.

7. Push, Pull et Fetch / Push, Pull and Fetch

50. FR - Que fait `git push` ? <code>git push</code> Réponse : Il envoie les nouveaux commits locaux vers le remote/upstream configuré.	EN - What does `git push` do? <code>git push</code> Answer: It sends new local commits to the configured remote/upstream.
51. FR - Que fait `git pull` ? <code>git pull</code> Réponse : Il récupère les modifications du serveur puis les intègre dans la branche locale selon la configuration.	EN - What does `git pull` do? <code>git pull</code> Answer: It fetches remote changes and integrates them into the local branch according to configuration.
52. FR - Que fait `git fetch` ? <code>git fetch</code> Réponse : Il télécharge les nouvelles références et objets du serveur sans fusionner automatiquement dans la branche courante.	EN - What does `git fetch` do? <code>git fetch</code> Answer: It downloads new references and objects from the server without automatically merging them into the current branch.
53. FR - Quelle différence principale entre `fetch` et `pull` ? Réponse : `fetch` récupère seulement ; `pull` récupère puis intègre.	EN - What is the main difference between `fetch` and `pull`? Answer: `fetch` only retrieves; `pull` retrieves and then integrates.
54. FR - Que fait `git fetch --prune` ? <code>git fetch --prune</code> Réponse : Il met à jour les références distantes et supprime localement les références de branches distantes qui n'existent plus.	EN - What does `git fetch --prune` do? <code>git fetch --prune</code> Answer: It updates remote references and removes local remote-tracking references for remote branches that no longer exist.
55. FR - ★ Que signifie `origin/main` ? Réponse : C'est la référence locale qui suit l'état connu de la branche `main` sur le remote `origin`.	EN - ★ What does `origin/main` mean? Answer: It is the local remote-tracking reference for the known state of `main` on remote `origin`.
56. FR - ★ Pourquoi faire `git fetch` avant une comparaison ou un rebase ? Réponse : Pour travailler avec une vue récente de l'état du serveur sans modifier immédiatement la branche locale.	EN - ★ Why run `git fetch` before a comparison or rebase? Answer: To work with a recent view of the server state without immediately changing the local branch.

8. Annulation, restore, reset, revert et reflog / Undo, Restore, Reset, Revert, and Reflog

57. FR - Que fait `git restore <fichier>` ? <code>git restore main.c</code> Réponse : Il annule les modifications non committées du fichier en le restaurant depuis l'index ou le commit selon le cas.	EN - What does `git restore <file>` do? <code>git restore main.c</code> Answer: It discards uncommitted changes to the file by restoring it from the index or commit as applicable.
58. FR - Que fait `git restore --staged <fichier>` ? <code>git restore --staged main.c</code> Réponse : Il retire le fichier de la Staging Area sans supprimer ses modifications du working tree.	EN - What does `git restore --staged <file>` do? <code>git restore --staged main.c</code> Answer: It removes the file from the Staging Area without deleting its working-tree changes.
59. FR - Que fait `git reset --soft HEAD~1` ? <code>git reset --soft HEAD~1</code> Réponse : Il déplace la branche avant le dernier commit mais conserve les modifications dans l'index.	EN - What does `git reset --soft HEAD~1` do? <code>git reset --soft HEAD~1</code> Answer: It moves the branch back before the last commit while keeping the changes staged.
60. FR - Que fait `git reset --hard HEAD~1` ? <code>git reset --hard HEAD~1</code> Réponse : Il recule le dernier commit et remplace index/working tree ; les modifications locales non sauvegardées peuvent être perdues.	EN - What does `git reset --hard HEAD~1` do? <code>git reset --hard HEAD~1</code> Answer: It moves back one commit and resets the index/working tree; unsaved local changes can be lost.
61. FR - Pourquoi `git reset --hard` demande-t-il de la prudence ? Réponse : Il peut effacer des changements locaux non committés. Il faut vérifier l'état et savoir ce qui sera perdu.	EN - Why must `git reset --hard` be used carefully? Answer: It can delete uncommitted local changes. You should verify the state and know what will be lost.

62. FR - Que fait `git revert <hash>` ? <code>git revert <commit_hash></code> Réponse : Il crée un nouveau commit qui inverse les changements du commit ciblé sans réécrire l'historique existant.	EN - What does `git revert <hash>` do? <code>git revert <commit_hash></code> Answer: It creates a new commit that reverses the target commit without rewriting existing history.
--	--

63. FR - Pourquoi `revert` est-il généralement plus sûr sur une branche partagée ? Réponse : Parce qu'il ajoute un nouveau commit au lieu de déplacer l'historique déjà publié.	EN - Why is `revert` generally safer on a shared branch? Answer: Because it adds a new commit instead of moving already published history.
--	---

64. FR - Que fait `git reflog` ? <code>git reflog</code> Réponse : Il affiche les déplacements récents de références locales, utiles pour retrouver des commits après reset ou rebase.	EN - What does `git reflog` do? <code>git reflog</code> Answer: It shows recent local reference movements, useful for recovering commits after reset or rebase.
--	---

65. FR - Comment revenir à un état retrouvé dans le reflog ? <code>git reset --hard <reflog_hash></code> Réponse : Après vérification, on peut utiliser le hash correspondant avec une commande adaptée, par exemple reset.	EN - How can you return to a state found in reflog? <code>git reset --hard <reflog_hash></code> Answer: After verification, use the corresponding hash with an appropriate command, such as reset.
---	--

9. Suppression de fichiers, branches et suivi / Deleting Files, Branches, and Tracking

66. FR - ★ Comment supprimer un fichier suivi par Git ? <code>git rm obsolete.txt</code> Réponse : Avec `git rm fichier`, puis commit.	EN - ★ How do you delete a Git-tracked file? <code>git rm obsolete.txt</code> Answer: Use `git rm file`, then commit.
--	---

67. FR - ★ Comment arrêter de suivre un fichier sans l'effacer du disque ? <code>git rm --cached config.local</code> Réponse : Avec `git rm --cached fichier`, puis l'ajouter éventuellement à `.gitignore`.	EN - ★ How do you stop tracking a file without deleting it from disk? <code>git rm --cached config.local</code> Answer: Use `git rm --cached file`, then optionally add it to `.gitignore`.
--	---

68. FR - ★ Comment supprimer une branche locale déjà fusionnée ? <code>git branch -d feature_uart</code> Réponse : Avec `git branch -d branche`.	EN - ★ How do you delete a merged local branch? <code>git branch -d feature_uart</code> Answer: Use `git branch -d branch`.
--	---

69. FR - ★ Que fait `git branch -D branche` ? <code>git branch -D feature_uart</code> Réponse : Il force la suppression locale même si Git considère que la branche n'est pas complètement fusionnée.	EN - ★ What does `git branch -D branch` do? <code>git branch -D feature_uart</code> Answer: It forces local deletion even if Git considers the branch not fully merged.
---	---

70. FR - ★ Comment supprimer une branche distante ? <code>git push origin --delete feature_uart</code> Réponse : Avec `git push origin --delete branche`.	EN - ★ How do you delete a remote branch? <code>git push origin --delete feature_uart</code> Answer: Use `git push origin --delete branch`.
---	---

71. FR - ★ À quoi sert `.gitignore` ? Réponse : Il indique à Git quels fichiers non suivis doivent être ignorés, par exemple builds, logs ou fichiers locaux.	EN - ★ What is `.gitignore` used for? Answer: It tells Git which untracked files should be ignored, such as builds, logs, or local files.
--	--

72. FR - ★ Est-ce que `.gitignore` arrête automatiquement de suivre un fichier déjà commité ? Réponse : Non. Il faut d'abord retirer le fichier du suivi, par exemple avec `git rm --cached`.	EN - ★ Does `.gitignore` automatically stop tracking an already committed file? Answer: No. You must first remove it from tracking, for example with `git rm --cached`.
--	--

10. Rebase et Cherry-pick / Rebase and Cherry-pick

73. FR - Que fait `git rebase main` ? <code>git rebase main</code> Réponse : Il rejoue les commits de la branche courante au-dessus de la pointe de `main`, créant un historique plus linéaire.	EN - What does `git rebase main` do? <code>git rebase main</code> Answer: It replays the current branch commits on top of `main`, creating a more linear history.
---	---

74. FR - ★ Pourquoi éviter de rebaser sans coordination des commits déjà partagés ? Réponse : Le rebase réécrit les identifiants de commits et peut perturber les autres utilisateurs de la branche.	EN - ★ Why avoid rebasing already shared commits without coordination? Answer: Rebase rewrites commit IDs and can disrupt other users of the branch.
---	---

75. FR - Que fait `git rebase -i HEAD~N` ? git rebase -i HEAD~5 Réponse : Il ouvre un rebase interactif des N derniers commits pour les réorganiser ou modifier.	EN - What does `git rebase -i HEAD~N` do? git rebase -i HEAD~5 Answer: It opens an interactive rebase for the last N commits so they can be reorganized or edited.
--	--

76. FR - Que signifie `squash` dans un rebase interactif ? Réponse : Fusionner un commit avec le précédent pour réduire plusieurs petits commits.	EN - What does `squash` mean in an interactive rebase? Answer: Combine a commit with the previous one to reduce several small commits.
--	---

77. FR - Que signifie `reword` ? Réponse : Conserver le contenu du commit mais modifier son message.	EN - What does `reword` mean? Answer: Keep the commit content but edit its message.
---	--

78. FR - Que signifie `drop` ? Réponse : Supprimer le commit de l'historique réécrit.	EN - What does `drop` mean? Answer: Remove the commit from the rewritten history.
--	--

79. FR - Que fait `git cherry-pick <hash>` ? git cherry-pick <commit_hash> Réponse : Il applique les changements d'un commit spécifique sur la branche courante en créant un nouveau commit.	EN - What does `git cherry-pick <hash>` do? git cherry-pick <commit_hash> Answer: It applies the changes from a specific commit to the current branch, creating a new commit.
--	---

80. FR - Quand cherry-pick est-il utile ? Réponse : Par exemple pour appliquer un correctif précis d'une branche à une autre sans fusionner toute la branche.	EN - When is cherry-pick useful? Answer: For example, to apply one specific fix from another branch without merging the entire branch.
--	---

11. Stash et travail interrompu / Stash and Interrupted Work

81. FR - Que fait `git stash` ? git stash Réponse : Il met temporairement de côté les modifications non committées afin de retrouver un working tree plus propre.	EN - What does `git stash` do? git stash Answer: It temporarily stores uncommitted changes to give you a cleaner working tree.
---	--

82. FR - Que fait `git stash pop` ? git stash pop Réponse : Il restaure le dernier stash puis le retire de la liste si l'application réussit.	EN - What does `git stash pop` do? git stash pop Answer: It restores the latest stash and removes it from the list if application succeeds.
---	---

83. FR - Que fait `git stash apply` ? git stash apply Réponse : Il applique un stash sans le supprimer de la liste.	EN - What does `git stash apply` do? git stash apply Answer: It applies a stash without removing it from the stash list.
---	--

84. FR - Que fait `git stash list` ? git stash list Réponse : Il affiche les stashes disponibles.	EN - What does `git stash list` do? git stash list Answer: It displays available stashes.
---	---

85. FR - Quand utiliser stash ? Réponse : Quand on doit interrompre rapidement un travail non terminé pour changer de tâche ou de branche.	EN - When do you use stash? Answer: When you need to quickly interrupt unfinished work to switch tasks or branches.
---	--

86. FR - ★ Pourquoi ne pas utiliser stash comme stockage long terme ? Réponse : Les stashes sont moins visibles qu'une vraie branche et sont mieux adaptés à du travail temporaire.	EN - ★ Why not use stash as long-term storage? Answer: Stashes are less visible than a real branch and are better suited to temporary work.
--	--

12. Tags, versions et releases / Tags, Versions, and Releases

87. FR - ★ À quoi sert un tag Git ? Réponse : À donner un nom stable à un commit, souvent pour marquer une release ou un jalon.	EN - ★ What is a Git tag used for? Answer: To give a stable name to a commit, often marking a release or milestone.
88. FR - Que fait `git tag -a v2.1.0 -m '...'` ? <code>git tag -a v2.1.0 -m "Release PCBA v2.1.0"</code> Réponse : Il crée un tag annoté avec message associé.	EN - What does `git tag -a v2.1.0 -m '...'` do? <code>git tag -a v2.1.0 -m "Release PCBA v2.1.0"</code> Answer: It creates an annotated tag with an associated message.
89. FR - Que fait `git tag -s ...` ? <code>git tag -s v2.1.0 -m "Signed release"</code> Réponse : Il crée un tag signé cryptographiquement si la configuration de signature est disponible.	EN - What does `git tag -s ...` do? <code>git tag -s v2.1.0 -m "Signed release"</code> Answer: It creates a cryptographically signed tag if signing is configured.
90. FR - Que fait `git push origin --tags` ? <code>git push origin --tags</code> Réponse : Il pousse tous les tags locaux vers le remote `origin`.	EN - What does `git push origin --tags` do? <code>git push origin --tags</code> Answer: It pushes all local tags to remote `origin`.
91. FR - Que fait `git tag -l` ? <code>git tag -l</code> Réponse : Il liste les tags disponibles localement.	EN - What does `git tag -l` do? <code>git tag -l</code> Answer: It lists locally available tags.
92. FR - Que fait `git describe --tags --always` ? <code>git describe --tags --always</code> Réponse : Il produit une chaîne descriptive basée sur le tag le plus proche et le commit, utile pour versionner un build.	EN - What does `git describe --tags --always` do? <code>git describe --tags --always</code> Answer: It produces a descriptive string based on the nearest tag and commit, useful for build versioning.
93. FR - Pourquoi les tags sont-ils utiles en production hardware/firmware ? Réponse : Ils permettent de relier un livrable, un firmware ou un package de test à un commit exact.	EN - Why are tags useful in hardware/firmware production? Answer: They tie a deliverable, firmware, or test package to an exact commit.

13. Submodules / Git Submodules

94. FR - ★ Qu'est-ce qu'un Git submodule ? Réponse : C'est une référence d'un dépôt Git vers un commit précis d'un autre dépôt intégré comme sous-répertoire.	EN - ★ What is a Git submodule? Answer: It is a repository reference to a specific commit of another repository integrated as a subdirectory.
95. FR - Pourquoi utiliser un submodule en firmware ? Réponse : Pour intégrer une bibliothèque externe ou interne tout en conservant son historique et sa version indépendants.	EN - Why use a submodule in firmware? Answer: To integrate an external or internal library while keeping its history and version independent.
96. FR - Que fait `git submodule add <URL> <path>` ? <code>git submodule add <repo_URL> libs/driver</code> Réponse : Il ajoute un dépôt externe comme submodule au chemin choisi.	EN - What does `git submodule add <URL> <path>` do? <code>git submodule add <repo_URL> libs/driver</code> Answer: It adds an external repository as a submodule at the selected path.
97. FR - Que fait `git submodule update --init --recursive` ? <code>git submodule update --init --recursive</code> Réponse : Il initialise et checkout les submodules, y compris les submodules imbriqués.	EN - What does `git submodule update --init --recursive` do? <code>git submodule update --init --recursive</code> Answer: It initializes and checks out submodules, including nested submodules.
98. FR - Pourquoi cette commande est-elle souvent nécessaire après clone ? Réponse : Le clone du dépôt principal ne checkout pas automatiquement tout le contenu des submodules sans option/configuration appropriée.	EN - Why is this often needed after clone? Answer: Cloning the main repository does not automatically check out all submodule contents without the appropriate option/configuration.

99. FR - Que fait `git submodule update --remote --merge` ? <code>git submodule update --remote --merge</code> Réponse : Il met les sous-modules à jour depuis leur branche distante configurée puis intègre la nouvelle révision selon ce mode.	EN - What does `git submodule update --remote --merge` do? <code>git submodule update --remote --merge</code> Answer: It updates submodules from their configured remote branch and integrates the new revision using merge mode.
--	---

14. Debug historique : blame et bisect / Historical Debugging: Blame and Bisect

100. FR - Que fait `git blame <fichier>` ? <code>git blame main.c</code> Réponse : Il indique pour chaque ligne quel commit l'a modifiée en dernier, avec auteur et informations associées.	EN - What does `git blame <file>` do? <code>git blame main.c</code> Answer: It shows, for each line, the commit that last modified it along with author and related information.
---	--

101. FR - ★ Pourquoi `git blame` ne doit-il pas servir à 'blâmer' une personne ? Réponse : Son intérêt est de trouver le contexte historique d'une ligne et le commit associé, pas de désigner un coupable.	EN - ★ Why should `git blame` not be used to blame a person? Answer: Its purpose is to find the historical context of a line and its commit, not to assign fault.
--	--

102. FR - À quoi sert `git bisect` ? Réponse : À rechercher par dichotomie le commit qui a introduit une régression entre un bon et un mauvais état connus.	EN - What is `git bisect` used for? Answer: To binary-search for the commit that introduced a regression between known good and bad states.
--	--

103. FR - Que fait `git bisect start` ? <code>git bisect start</code> Réponse : Il démarre une session de bisect.	EN - What does `git bisect start` do? <code>git bisect start</code> Answer: It starts a bisect session.
---	---

104. FR - Que signifie `git bisect bad` ? <code>git bisect bad</code> Réponse : Il marque le commit courant comme défectueux.	EN - What does `git bisect bad` mean? <code>git bisect bad</code> Answer: It marks the current commit as bad.
---	---

105. FR - Que signifie `git bisect good <hash>` ? <code>git bisect good <old_good_hash></code> Réponse : Il indique un commit connu comme fonctionnel.	EN - What does `git bisect good <hash>` mean? <code>git bisect good <old_good_hash></code> Answer: It marks a known-good commit.
--	--

106. FR - Que fait `git bisect reset` ? <code>git bisect reset</code> Réponse : Il termine le bisect et restaure la position de travail initiale.	EN - What does `git bisect reset` do? <code>git bisect reset</code> Answer: It ends the bisect session and restores the initial working position.
---	---

15. Nettoyage, maintenance et intégrité / Cleanup, Maintenance, and Integrity

107. FR - Que fait `git clean -fdn` ? <code>git clean -fdn</code> Réponse : Il simule la suppression des fichiers/dossiers non suivis sans les supprimer réellement.	EN - What does `git clean -fdn` do? <code>git clean -fdn</code> Answer: It previews deletion of untracked files/directories without actually deleting them.
--	---

108. FR - Pourquoi exécuter `git clean -fdn` avant `git clean -fd` ? Réponse : Pour vérifier exactement ce qui serait supprimé.	EN - Why run `git clean -fdn` before `git clean -fd`? Answer: To verify exactly what would be deleted.
--	---

109. FR - Que fait `git clean -fd` ? <code>git clean -fd</code> Réponse : Il supprime les fichiers et dossiers non suivis correspondant aux options.	EN - What does `git clean -fd` do? <code>git clean -fd</code> Answer: It deletes untracked files and directories covered by the options.
--	--

110. FR - Que fait `git gc --prune=now` ? <code>git gc --prune=now</code> Réponse : Il lance immédiatement le garbage collection et peut supprimer des objets devenus inaccessibles selon l'état du dépôt.	EN - What does `git gc --prune=now` do? <code>git gc --prune=now</code> Answer: It immediately runs garbage collection and may remove unreachable objects depending on repository state.
--	--

111. FR - Que fait `git fsck` ? <code>git fsck</code> Réponse : Il vérifie la connectivité et l'intégrité des objets du dépôt et peut signaler des objets inaccessibles.	EN - What does `git fsck` do? <code>git fsck</code> Answer: It checks repository object connectivity and integrity and can report unreachable objects.
112. FR - Que fait `git count-objects -v` ? <code>git count-objects -v</code> Réponse : Il affiche des statistiques sur les objets et la taille de stockage du dépôt.	EN - What does `git count-objects -v` do? <code>git count-objects -v</code> Answer: It displays object and storage statistics for the repository.
113. FR - ★ Pourquoi les commandes de purge agressive sont-elles dangereuses ? Réponse : Elles peuvent rendre irrécupérables des objets qui auraient encore pu être retrouvés avec <code>reflog</code> ou <code>fsck</code> .	EN - ★ Why are aggressive pruning commands dangerous? Answer: They can make objects unrecoverable that might otherwise have been found with <code>reflog</code> or <code>fsck</code> .

16. Git Hooks / Git Hooks

114. FR - Qu'est-ce qu'un Git hook ? Réponse : Un script exécuté automatiquement lors de certains événements Git dans le dépôt local.	EN - What is a Git hook? Answer: A script automatically executed on certain Git events in the local repository.
115. FR - À quoi sert `pre-commit` ? Réponse : À exécuter des contrôles avant de créer un commit, par exemple <code>lint</code> ou tests rapides.	EN - What is `pre-commit` used for? Answer: To run checks before creating a commit, such as linting or quick tests.
116. FR - À quoi sert `commit-msg` ? Réponse : À valider ou imposer un format de message de commit.	EN - What is `commit-msg` used for? Answer: To validate or enforce a commit-message format.
117. FR - À quoi sert `pre-push` ? Réponse : À lancer des vérifications avant l'envoi de commits vers le serveur.	EN - What is `pre-push` used for? Answer: To run checks before commits are sent to the server.
118. FR - ★ Pourquoi un hook local ne remplace-t-il pas une CI centrale ? Réponse : Les hooks locaux peuvent être absents ou contournés ; la CI sur serveur fournit un contrôle commun et reproductible.	EN - ★ Why does a local hook not replace centralized CI? Answer: Local hooks may be missing or bypassed; server-side CI provides common, reproducible checks.
119. FR - ★ Donnez un exemple de hook utile en firmware. Réponse : Lancer un formater, un linter, des unit tests ou vérifier qu'un fichier généré requis est à jour.	EN - ★ Give an example of a useful firmware hook. Answer: Run a formatter, linter, unit tests, or verify that a required generated file is up to date.

17. Git LFS et Worktree / Git LFS and Worktree

120. FR - Pourquoi Git standard est-il moins pratique pour de gros fichiers binaires ? Réponse : Chaque nouvelle version binaire peut augmenter fortement la taille du dépôt car Git ne les diff pas aussi efficacement que du texte.	EN - Why is standard Git less convenient for large binary files? Answer: Each new binary version can significantly increase repository size because Git cannot diff them as efficiently as text.
121. FR - À quoi sert Git LFS ? Réponse : Il stocke de gros fichiers via un mécanisme LFS et laisse des pointeurs dans le dépôt Git principal.	EN - What is Git LFS used for? Answer: It stores large files through LFS and leaves pointer files in the main Git repository.
122. FR - Que fait `git lfs install` ? <code>git lfs install</code> Réponse : Il configure Git LFS pour l'utilisateur/dépôt selon l'environnement.	EN - What does `git lfs install` do? <code>git lfs install</code> Answer: It configures Git LFS for the user/repository depending on the environment.

123. FR - Que fait `git lfs track '*.step` ? <code>git lfs track "*.step"</code> Réponse : Il configure Git LFS pour suivre les fichiers STEP correspondant au pattern et modifie généralement <code>.gitattributes</code> .	EN - What does `git lfs track '*.step` do? <code>git lfs track "*.step"</code> Answer: It configures Git LFS to track STEP files matching the pattern and generally updates <code>.gitattributes</code> .
--	---

124. FR - Que fait `git lfs ls-files` ? <code>git lfs ls-files</code> Réponse : Il liste les fichiers du dépôt actuellement gérés par LFS.	EN - What does `git lfs ls-files` do? <code>git lfs ls-files</code> Answer: It lists repository files currently managed by LFS.
--	---

125. FR - Qu'est-ce qu'un Git worktree ? Réponse : Un checkout supplémentaire lié au même dépôt, permettant de travailler sur plusieurs branches dans des dossiers différents.	EN - What is a Git worktree? Answer: An additional checkout linked to the same repository, allowing work on multiple branches in different folders.
---	--

126. FR - Que fait `git worktree add ../hotfix-folder main` ? <code>git worktree add ../hotfix-folder main</code> Réponse : Il crée un nouveau working tree dans <code>../hotfix-folder</code> basé sur la branche <code>main</code> si la situation de branche le permet.	EN - What does `git worktree add ../hotfix-folder main` do? <code>git worktree add ../hotfix-folder main</code> Answer: It creates another working tree in <code>../hotfix-folder</code> based on <code>main</code> when branch constraints allow it.
--	---

18. Scénarios pratiques d'entretien / Practical Interview Scenarios

127. FR - ★ Vous avez modifié un fichier mais n'avez pas fait `git add`. Comment voir les changements ? <code>git diff</code> Réponse : Avec <code>git diff</code> .	EN - ★ You modified a file but have not run `git add`. How do you see the changes? <code>git diff</code> Answer: Use <code>git diff</code> .
--	--

128. FR - ★ Vous avez fait `git add` par erreur mais voulez conserver le fichier modifié. Que faire ? <code>git restore --staged file.c</code> Réponse : Utiliser <code>git restore --staged <fichier></code> .	EN - ★ You ran `git add` by mistake but want to keep the file modified. What do you do? <code>git restore --staged file.c</code> Answer: Use <code>git restore --staged <file></code> .
---	---

129. FR - ★ Un mauvais commit est déjà poussé sur une branche partagée. `reset` ou `revert` ? Réponse : En général <code>revert</code> , car il annule par un nouveau commit sans réécrire l'historique partagé.	EN - ★ A bad commit has already been pushed to a shared branch. `reset` or `revert`? Answer: Generally <code>revert</code> , because it undoes the change with a new commit without rewriting shared history.
---	--

130. FR - ★ Vous devez corriger rapidement un bug sur `main` sans perdre votre travail actuel. Quelles options ? Réponse : Créer un worktree séparé, ou temporairement stash les changements puis changer de branche.	EN - ★ You need to fix a bug on `main` without losing current work. What are your options? Answer: Create a separate worktree, or temporarily stash the changes and switch branches.
--	---

131. FR - ★ Après `git pull`, vous obtenez un conflit. Que faites-vous ? Réponse : Identifier les fichiers en conflit, résoudre le contenu, tester, stage les résolutions puis terminer l'intégration.	EN - ★ After `git pull`, you get a conflict. What do you do? Answer: Identify conflicted files, resolve the content, test, stage the resolutions, then complete the integration.
---	---

132. FR - ★ Vous cherchez le commit qui a introduit un bug intermittent apparu récemment. Quel outil Git peut aider ? Réponse : <code>git bisect</code> , si vous pouvez identifier un état good et un état bad reproductibles.	EN - ★ You are looking for the commit that introduced a recently appearing bug. Which Git tool can help? Answer: <code>git bisect</code> , if you can identify reproducible good and bad states.
--	---

133. FR - ★ Un fichier STEP de 200 MB change souvent. Que considérer ? Réponse : Git LFS peut être préférable à stocker chaque version directement comme binaire standard dans Git.	EN - ★ A 200 MB STEP file changes often. What should you consider? Answer: Git LFS may be preferable to storing every version directly as a standard Git binary.
--	---

134. FR - ★ Comment relier une release firmware livrée à la production à Git ? Réponse : Créer un tag de release sur le commit exact et enregistrer cette version dans le firmware ou le rapport de build.	EN - ★ How do you tie a production firmware release to Git? Answer: Create a release tag on the exact commit and record that version in the firmware or build report.
---	--

135. FR - ★ Vous avez supprimé un commit local par reset mais vous vous souvenez qu'il existait. Que vérifier d'abord ? git reflog Réponse : `git reflog` pour retrouver le hash avant toute nouvelle purge.	EN - ★ You removed a local commit with reset but remember it existed. What do you check first? git reflog Answer: `git reflog` to recover the hash before any new pruning.
136. FR - ★ Quelle commande utilisez-vous presque toujours avant un commit important ? Réponse : `git status`, puis souvent `git diff` et `git diff --staged` pour vérifier exactement ce qui sera enregistré.	EN - ★ Which command do you almost always use before an important commit? Answer: `git status`, often followed by `git diff` and `git diff --staged` to verify exactly what will be recorded.
137. FR - ★ Que fait cette séquence ? git status git add . git commit -m "Fix UART timeout" git push Réponse : Elle vérifie l'état, stage tous les changements du dossier courant, crée un commit puis le pousse vers l'upstream.	EN - ★ What does this sequence do? git status git add . git commit -m "Fix UART timeout" git push Answer: It checks status, stages current-directory changes, creates a commit, then pushes it to the upstream.
138. FR - ★ Que fait cette séquence ? git fetch --prune git log --oneline Réponse : Elle récupère les références distantes sans intégrer, puis affiche l'historique compact pour inspection.	EN - ★ What does this sequence do? git fetch --prune git log --oneline Answer: It retrieves remote references without integrating them, then displays compact history for inspection.